Mention four architectural levels of computer systems.

列出计算机系统的四个架构级别。

Digital Logic Level: Handles logic gates and basic circuits.

数字逻辑层：处理逻辑门和基本电路。

Microarchitecture Level: Deals with the design of CPU components such as cache and memory controllers.

微架构层：涉及 CPU 组件（如缓存和内存控制器）的设计。

Instruction Set Architecture (ISA) Level: Defines the set of instructions that a CPU can execute.

指令集架构层（ISA）：定义 CPU 可以执行的指令集。

System Level: Combines the hardware and software to create the overall system functionality.

系统层：结合硬件和软件来实现整体系统功能。

What does CPU stand for? ALU?

CPU 和 ALU 分别代表什么？

CPU stands for Central Processing Unit, which is responsible for executing instructions and performing calculations.

CPU 代表 中央处理器（Central Processing Unit），负责执行指令和进行计算。

ALU stands for Arithmetic Logic Unit, a part of the CPU that handles arithmetic and logical operations.

ALU 代表 算术逻辑单元（Arithmetic Logic Unit），它是 CPU 的一部分，负责算术和逻辑操作。

What are the components of a computer system?

计算机系统的组成部分有哪些？

CPU (Central Processing Unit): Executes instructions.

中央处理器（CPU）：执行指令。

Memory (RAM): Stores data temporarily.

内存（RAM）：临时存储数据。

Storage (e.g., hard drives): Stores data permanently.

存储设备（如硬盘）：永久存储数据。

Input/Output Devices: Devices like keyboards, mice, and monitors.

输入/输出设备：如键盘、鼠标和显示器。

Mention 4 different types of computers.

列出四种不同类型的计算机。

Types of computers include:

计算机的类型包括：

Mainframe computers: Large, powerful computers used by businesses.

大型计算机：企业使用的强大计算机。

Supercomputers: Used for complex scientific computations.

超级计算机：用于复杂的科学计算。

Personal Computers (PCs): Used by individuals for daily tasks.

个人计算机（PCs）：个人用于日常任务。

Embedded Systems: Computers embedded in other devices like cars and appliances.

嵌入式系统：嵌入到其他设备中的计算机，如汽车和家电。

‘Servers’ are facilitated via the availability of ?

‘服务器’的实现依赖于什么？

- Servers rely on reliable network infrastructure and high-performance hardware to serve multiple users or clients simultaneously.

服务器依赖于可靠的网络基础设施和高性能硬件来同时为多个用户或客户端提供服务。

Define the areas of study for ‘computer systems’.

定义‘计算机系统’的研究领域。

Computer Architecture: Structure and design of computer components.

计算机架构：计算机组件的结构和设计。

Operating Systems: Software that manages hardware and software resources.

操作系统：管理硬件和软件资源的软件。

Networking: Communication between different computer systems.

网络：不同计算机系统之间的通信。

Data Storage: How information is stored, processed, and retrieved.

数据存储：信息如何存储、处理和检索。

Define ‘Downward Compatibility’.

定义‘向下兼容’。

- Downward Compatibility, also known as Backward Compatibility, means that newer systems or software can operate with older hardware or software without any problems.

向下兼容（也称为向后兼容）指的是新系统或软件能够与旧硬件或软件一起正常运行而不会出现问题。

What is the description language that can be used to design high-speed IC hardware?

用于设计高速集成电路（IC）硬件的描述语言是什么？

- VHDL (Very High-Speed Integrated Circuit Hardware Description Language) is used for designing high-speed IC hardware.

VHDL（甚高速集成电路硬件描述语言）用于设计高速集成电路硬件。

1.What are the advantages of having a hierarchical approach to computer system?

采用计算机系统分层结构的优势是什么？

- A hierarchical approach simplifies the design and understanding of complex systems by breaking them down into smaller, more manageable subsystems. Each layer interacts only with the layers directly above and below it, making debugging, maintenance, and system upgrades easier.

- 分层结构可以简化复杂系统的设计和理解，通过将系统分解为更易管理的子系统。每一层只与直接上下层交互，这使得调试、维护和系统升级更加容易。

What is Moore’s Law?

摩尔定律是什么？

- Moore’s Law states that the number of transistors on a microchip doubles approximately every two years, leading to the continuous increase in computing power and decrease in relative cost.

• 摩尔定律指出，集成电路上晶体管的数量大约每两年翻一番，导致计算能力的持续提升和相对成本的下降。

3. Functionalities of hardware systems can be brought out by what and thus offered to the user?

硬件系统的功能通过什么实现并提供给用户？

• The functionalities of hardware systems are enabled by software, specifically operating systems and application programs, which allow users to interact with the hardware efficiently.

• 硬件系统的功能通过软件实现，尤其是操作系统和应用程序，使用户能够高效地与硬件交互。

4. What are the advantages of having ‘operating systems’ wrapping around the computer hardware?

让‘操作系统’围绕计算机硬件的优势是什么？

• Operating systems abstract the complexity of hardware, providing a simpler and more user-friendly interface. They manage resources such as memory, CPU, and input/output devices, ensuring that multiple applications can run efficiently and securely.

• 操作系统抽象了硬件的复杂性，提供了一个更简单、用户友好的界面。它们管理内存、CPU 和输入/输出设备等资源，确保多个应用程序能够高效、安全地运行。

5. WIMP stand for? WIMP is available due to the development of?

WIMP 代表什么？WIMP 的实现得益于什么的发展？

• WIMP stands for Windows, Icons, Menus, and Pointers. It became available due to the development of graphical user interfaces (GUIs), which allow users to interact with computers using visual elements.

• WIMP 代表窗口、图标、菜单和指针。它的实现得益于图形用户界面（GUI）的发展，使用户可以通过可视化元素与计算机交互。

6. What is the focus of scientific computing?

科学计算的重点是什么？

• The focus of scientific computing is solving complex mathematical models and simulations, typically requiring high-performance computing to process large data sets, run simulations, or solve numerical problems.

• 科学计算的重点是解决复杂的数学模型和模拟，通常需要高性能计算来处理大量数据集、运行模拟或解决数值问题。

7. What is the focus of business computing?

商业计算的重点是什么？

• Business computing focuses on managing and processing business data efficiently, improving decision-making processes, and optimizing business operations. It often involves databases, enterprise software, and financial systems.

• 商业计算的重点是高效管理和处理业务数据，改进决策过程，优化业务运营。它通常涉及数据库、企业软件和财务系统。

8. What is the major characteristic of personal computing?

个人计算的主要特征是什么？

• The major characteristic of personal computing is providing users with accessible and interactive computing resources tailored to individual needs, typically through devices like PCs, tablets, or smartphones.

• 个人计算的主要特征是为用户提供易于访问的交互式计算资源，通常通过个人电脑、平板电脑或智能手机等设备满足个人需求。

输入-处理-输出模型（Input-Process-Output Model）

• 定义：这是描述计算机系统如何处理数据的基本模型。输入（数据被引入系统）、处理（数据被 CPU 处理）、输出（处理后的数据输出）是这一模型的三个基本部分。

输入-处理-输出模型（Input-Process-Output Model）

• 定义：这是一种计算机体系结构设计，其中程序指令存储和数据存储在同一内存系统中。它基于存储程序概念，即机器指令和数据共享同一内存空间。

哈佛架构（Harvard Architecture）

• 定义：与冯·诺依曼架构不同，哈佛架构拥有分开的存储和信号通路，为程序和数据提供独立的物理存储。这允许数据和指令的并行访问，可能提高系统的性能。

主要差异

• 冯·诺依曼与哈佛：主要差异在于存储和路径的分离。哈佛架构可以同时访问指令和数据，减少访问延迟，而冯·诺依曼架构由于使用单一内存空间，指令和数据的访问可能会冲突。

哈佛架构的优点和成本

- 优点：提高了处理速度，因为指令和数据的并行访问避免了瓶颈。
- 成本：更复杂的物理结构可能导致成本增加，因为需要额外的硬件来分离存储和管理不同的通道。

1. 哪部分计算机负责存储数据和程序供 CPU 访问？

- 答：主存储器（RAM）。

2. 二级存储的三个例子：硬盘驱动器（HDD）、固态驱动器（SSD）、光盘（CD/DVD）。

3. 输入设备的三个例子：键盘、鼠标、扫描仪。

4. 输出设备的三个例子：显示器、打印机、扬声器。

5. 软件的定义：软件是计算机程序的集合，包括所有的系统软件、应用软件和中间件，用于指挥计算机硬件执行特定的任务和功能。

给出四个高级编程语言的例子（Name 4 examples of HLLs.）：

- Java
- Python
- C++
- Ruby

翻译填补了计算机系统中的语义差距（Translation fills in the semantic gap in computer systems.）：

- 正确（True）。翻译将高级语言代码转换为机器可以执行的指令，帮助解决人类语言与计算机语言之间的理解差异。

列出三种翻译方式，并指出每种方式下翻译的关键角色（Name 3 different ways of translation. Identify the crucial role of translation under each.）：

- 编译（Compilation）：将源代码一次性转换成机器代码，优化执行速度。
- 解释（Interpretation）：逐行执行源代码，增强脚本灵活性和跨平台性。
- 汇编（Assembly）：将汇编语言转换为机器代码，提供对硬件操作的精细控制。

在编译时发生错误时我们怎么办（When compile-time errors occur, what do we do?）：

- 分析错误信息，修改源代码以解决错误。

链接的目的是什么（What are the purposes of linking?）：

- 将多个对象文件和库合并成一个单一的可执行文件，解析外部依赖关系。

‘加载’是在编译后‘链接’之前进行的（‘Loading’ is carried out before ‘linking’ after a program is compiled.）：

- 错误（False）。加载通常在链接后发生，将可执行文件加载到内存中以供执行。

程序模块可以单独编译（Program modules can be compiled separately.）：

- 正确（True）。这允许更灵活的开发和只重新编译修改过的模块。

编译器可以将模块转换成二进制代码，但它不能解决对其他模块的引用（A compiler can translate a module into binary codes, but it cannot resolve those references to other modules.）：

- 这种情况发生在模块间存在未解析的外部引用时，需要链接器在链接阶段解决。

如果链接到程序代码中，库文件就是可用的（Library files are usable if linked into your program code.）：

- 正确（True）。库文件需要被正确链接到程序中才能被调用和使用。

解释器通常将程序代码转换成什么（Interpreters typically convert program code into what?）：

- 解释器通常将程序代码转换为中间字节码或直接执行。

程序编译的输出是什么（What is the output of program compilation?）：

- 程序编译的输出通常是对象代码或字节码，取决于编译的目标语言和平台。

列出两种解释器比编译器更有用的场景（Name 2 scenarios where interpreters are more useful than compilers.）：

- 脚本执行和测试：快速迭代和动态执行代码。
- 教育和学习：即时反馈有助于教学和学习编程语言。

解释器有时被称为虚拟机的原因（Interpreters are sometimes called as virtual machines because … ?）：

- 因为它们在执行程序代码时，创建了一个抽象的执行环境，隔离了硬件和操作系统的具体细节。

提出三种代码共享的方法（Mention 3 approaches to code sharing.）：

- 使用静态库和动态库。
- 通过包管理器和依赖管理工具共享和安装代码。

- 利用网络 APIs 和微服务架构来共享功能。

列出两种宏库的缺点或问题（Name 2 disadvantages (or issues) of macro libraries.）：

- 宏在预处理期间展开，可能导致代码膨胀，增加编译时间和执行时间。
- 宏不提供类型安全，容易引发错误和调试困难。

库可以链接到您的程序代码中，但不能更改（Libraries can be linked into your program code, but not altered.）：

- 正确（True）。库提供了预编译的代码块，可以链接使用但通常不直接修改。

预转译程序库执行的程序执行的缺点是什么（What are the disadvantages of program execution with pre-translated program library?）：

- 可能导致程序的运行效率降低，因为库可能没有针对特定平台或情况进行优化。

动态库和动态链接存在事实上的标准（There exists a de-facto standard of dynamic libraries and dynamic linking.）：

- 正确（True）。动态链接库（DLLs）和其他形式的动态链接在多个平台上被广泛支持，形成了事实上的标准。

Define ‘bit’.

- “Bit” 是计算机中的最小数据单位，它是 Binary Digit（二进制数字）的缩写。一个 bit 只能取 0 或 1 这两个值。

Decimal notation is more compact than binary. (T or F)

- True（T）。十进制表示法比二进制更加紧凑。因为十进制的数字范围更大（0 到 9），而二进制只有 0 和 1，因此相同的数字用二进制表示时需要更多的位数。

Binary is more “convenient” for computers due to ?

- 二进制对计算机更方便是因为计算机的物理结构使用 开关电路，这与二进制的 开（1）和关（0） 状态非常匹配。因此，二进制最适合计算机硬件的处理方式。

Every number can be used as a base. (T or F)

- False（F）。虽然理论上任何数都可以作为基数（base），但实际上使用的基数是有限的，最常见的是二进制（基数 2）、十进制（基数 10）、八进制（基数 8）和十六进制（基数 16）。

Hexadecimal notation uses ? as a base.

- 十六进制（Hexadecimal）使用 16 作为基数。

Octal notation uses ? as a base.

- 八进制（Octal）使用 8 作为基数。

Why do we use hexadecimal, or base 16, number notation?

- 我们使用十六进制表示法是因为它比二进制更加简洁，但同时依然与计算机的二进制系统密切相关。每个十六进制数字可以表示 4 个二进制位（一个半字节），这使得它在表示内存地址或机器指令时更加直观和易于理解。

How many bits are required to encode ASCII?

- 7 位（bits）用于编码标准 ASCII 字符集。虽然 ASCII 编码标准使用 7 位表示，但通常会用 1 个字节（8 位）来存储每个字符，最后的 1 位通常保留或用于扩展。

最广泛使用的二进制编码:

- 对于非 IBM 的大型计算机和几乎所有的微型计算机来说，最广泛使用的二进制编码是 ASCII。

不常用的编码系统:

- 目前不常用的编码系统是 Unicode，虽然它广泛应用于现代计算机系统中支持多语言，但在这个选项中被视为不常用。

ASCII 的一个缺点:

- ASCII 的一个缺点是它不能处理英语和欧洲语言以外的某些语言的字符。

ASCII 被分为哪两类代码？:

- ASCII 被分为控制字符和可打印字符两大类。

最广泛使用的浮点计算标准:

- 最广泛使用的浮点计算标准是 IEEE 754。

声明变量时会发生什么:

- 声明变量时，系统会为变量分配内存空间，并可能初始化变量（取决于语言和上下文）。

1.What is the most important development during the past 40 years evolution of OS?

- 回答：过去 40 年操作系统最重要的发展之一是多任务处理（multi-tasking）的引入，以及网络功能的集成。这些改进大大增强了操作系统的功能，使其能够处理多个程序，同时允许计算机连接到全球网络。

2.OS needs to provide fair & protected access to system resources, why?

• 回答：操作系统需要提供公平和受保护的资源访问，以确保多个程序和用户可以同时使用系统，而不会相互干扰。此外，受保护的访问可以防止未授权的用户或程序破坏系统的安全性或稳定性。

3.Explain ‘multi-tasking’.

• 回答：多任务处理是指操作系统能够同时运行多个程序。这意味着操作系统可以在不影响其他任务的情况下为每个程序分配时间和资源，用户可以同时执行多项任务。

4.Mention 3 functions that need to be provided when an OS is to support ‘multi-tasking’.

• 回答：支持多任务处理的操作系统需要提供以下三项功能：

1. 进程管理：管理多个进程的创建、执行和终止。

2. 内存管理：确保每个任务都有足够的内存使用，并防止它们相互覆盖。

3. CPU 调度：分配 CPU 时间给每个任务，确保每个任务都有时间执行。

5.Explain ‘multi-user’.

• 回答：多用户功能指操作系统可以支持多个用户同时使用同一台计算机或系统资源。每个用户都有独立的账户、权限和资源分配，确保彼此之间的隔离性和安全性。

6.How does operating system provide access to network facilities?

•回答：操作系统通过网络 API（如套接字接口）提供对网络设施的访问。它管理网络通信，确保数据可以通过网络发送和接收，使应用程序能够与远程服务器进行交互。

7.Telecommuting or online shopping are enabled through the introduction of ?

• 回答：远程办公和在线购物是通过互联网的引入得以实现的。操作系统通过提供对网络的支持，使得用户可以通过互联网进行远程访问、购物、交流等活动。

General-purpose machine（通用计算机）与 Special-purpose machine（专用计算机）之间的区别：•

General-purpose machine（通用计算机）：

• 这是指像 CPU（中央处理器）这样的组件。它是可以执行多种任务的通用计算设备，能够处理多种不同类型的计算任务。CPU 是可以通过软件指令来执行各种任务的核心计算单元。

• Special-purpose machine（专用计算机）：

• 这是指协处理器（Coprocessor），它通常用于执行特定类型的任务，例如图形处理、浮点运算等。协处理器并不能独立完成所有计算任务，而是专门优化某些任务，

协助 CPU 进行特定类型的计算。

点对点连接下 6 个设备所需的线数计算：

- 题目假设有 6 个设备，并通过 8 条数据线和 2 条控制线连接。
- 点对点连接需要计算每一对设备之间的连接路径，路径数量计算公式是 n(n-1)/2 。
- 对于 6 个设备，路径数为 6(6-1)/2 。
- 每条路径需要 8 条数据线和 2 条控制线，因此每条路径需要 10 条线。
- 总共所需的线数为 15* 10 =150 条。

在每次指令执行过程中，IP（指令指针） 和 IR（指令寄存器） 是总是被使用的两个寄存器。

2 registers that are always used during each instruction execution.

IP（Instruction Pointer，指令指针）：

- IP 寄存器保存下一条即将执行的指令地址。每次取指时，IP 会被使用来确定从内存中获取的指令位置，之后 IP 会递增以准备下一条指令。

IR（Instruction Register，指令寄存器）：

- IR 寄存器存储从内存中取回的当前指令。每当 CPU 从内存中读取指令后，这条指令会被加载到 IR 寄存器中进行解码和执行。

列出两种由协处理器运行的典型应用程序：

你可以想到的两种常见的协处理器任务，比如图形处理和数学计算。

2. 列出计算机系统中的三种不同类型的总线：

常见的总线类型包括数据总线、地址总线和控制总线。

3. 列出导致总线瓶颈的两个原因：

总线瓶颈可能由于多设备共享相同的数据路径以及数据传输速度限制等因素导致。

4. 指出 CPU 的两个主要组件：

CPU 的主要组件通常是控制单元和算术逻辑单元（ALU）。

5. 列出在一个机器周期内执行的任务：

机器周期包括取指令、解码和执行等步骤。

6. 计算机和打印机通常通过什么类型的端口连接？

常见的连接类型包括 USB 端口和并行端口。

7. RISC 指的是：

选项 b. 支持比 CISC 芯片更少指令的处理器，这个解释是最准确的。

8. 计算机的主要处理器通过指令操作数据将其转换为信息，这被称为什么？

正确答案是 b. CPU，因为这是中央处理器的职责。

9. 这种类型的硬件由能够将数据翻译成计算机可以处理形式的设备组成，它是？

正确答案是 b. 输入设备。

10. 列出每次指令执行过程中总是使用的四个寄存器：

这些寄存器可能包括程序计数器（PC），指令寄存器（IR），地址寄存器（AR）和数据寄存器（DR）。

execute 阶段是否对所有指令类型都相同？

- 解释：在 CPU 执行指令时，不同类型的指令（如算术运算、跳转指令等）的执行阶段可能不同。例如，加法指令与跳转指令所执行的操作本质上不同，因此这句话是 False。

汇编器将汇编语言翻译成哪种形式？

- 解释：汇编器会将汇编语言转换为机器代码（Machine Code），这是一种 CPU 可以直接理解和执行的二进制指令。

汇编器将标签与什么关联？Assembler associates labels with

- 解释：汇编器将标签与内存地址关联。标签通常用于表示跳转或循环的目标位置，帮助程序找到相应的内存位置。

CPU 只能执行加载到主存储器（RAM）中的指令（True or False）？

- 解释：这是 True。CPU 只能执行那些已经被加载到 RAM 中的指令，因为 CPU 无法直接从磁盘等辅助存储器中读取和执行指令。

RAM 的所有存储单元的访问时间是否相同（True or False）？

- 解释：这是 True。RAM 中的所有单元都是随机访问的，意味着访问任意单元的时间是相同的，而不像顺序访问存储器（如磁带）那样依赖于数据的位置。

6.MOV EBX, [EBX] 指令的含义是什么？

- 解释：这条指令将 EBX 寄存器的值作为地址，从该地址处取出值并存入 EBX 寄存器中。因此，EBX 首先表示一个内存地址，然后将该内存地址中的值赋给自己。这种

操作类似于“解引用”指针。

- 副作用：该指令会覆盖掉 EBX 中的原始值，EBX 不再保存原本的地址，而是变成该地址处的内容。

7.JNZ L2 是什么意思？

- 解释：JNZ 是汇编中的“跳转如果不为零”（Jump if Not Zero）指令。它表示如果零标志（ZF）位没有被设置（即结果不为零），程序就会跳转到标签 L2 所指示的内存地址处。

- 编译错误：如果 L2 没有在代码中定义，编译器会报错，因为它找不到需要跳转的位置。

8.MOV maxval, loc1 是否为有效的汇编代码？

- 解释：这取决于 loc1 和 maxval 是否已被**定义为有效的内存地址或寄存器**。如果它们没有被正确定义，那么这条指令可能会报错，无法编译通过。

列出 Pentium 处理器中 Flag 寄存器的 3 个使用场景：

- Flag 寄存器可以在以下场景中使用：

条件跳转操作（例如，使用 CMP 指令后，通过零标志判断两个值是否相等）。

算术操作中检测溢出（通过溢出标志位检测溢出）。

中断处理（设置或清除中断标志来控制中断）。

Flag 寄存器可以用于在一条指令和后续指令之间传递信息（对或错？）：

- 答案：对。

哪一个寄存器用于存储此指令的减法结果？CMP AL, BL：

- 答案：AL 寄存器（AL 为目标寄存器，存储比较结果的标志位会反映在 Flag 寄存器中）。

在内联汇编中，如何传递参数给 printf：

- 这里的问题是关于在内联汇编中，如何将参数传递给 printf 函数，正确的方式是通过堆栈（stack）来传递参数，利用 push eax 等操作将值压入堆栈。printf 读取栈上的参数地址进行输出。

5. 内联汇编可以调用 C 库函数（True or False）：

- True。

状态标志在指令执行之前是否被设置：

- 实际上，状态标志是在指令执行之后根据结果进行更新的，False。

D 标志用于设置循环方向：

- D 标志通常用于控制字符串操作指令（如 MOVSB、STOSB）中的递增或递减方向。

1. The Pentium instruction set has a fixed width for all instructions to speed up instruction decoding.

这道题在询问 Pentium 指令集是否使用了固定宽度的指令来加速解码。

- 解答：错误（False）。Pentium 指令集的指令长度并不是固定的，它可以根据操作和寻址模式的不同在 1 到 15 个字节之间变化。虽然固定宽度的指令在某些处理器架构中（如 RISC 架构）有助于加快解码速度，但 Pentium 是基于复杂指令集（CISC）的架构，其指令长度可变。

2. Which sample program is more efficient? (slide 12 vs slide 14)

比较第 12 页和第 14 页的代码示例，哪一个更高效？

- 解答：第 12 页的代码使用的是寄存器间接寻址模式，而第 14 页使用的是带位移寄存器间接寻址模式。第 12 页的代码可能会更高效，因为它直接利用寄存器中的地址操作数据，而第 14 页的代码需要计算偏移量。寄存器间接寻址在某些情况下可以减少计算步骤，提高执行速度。

3. What are those 3 major components covered by every instruction?

每条指令都包含哪三个主要部分？

- 解答：

（1） 操作（Operation）：指定要执行的具体操作，比如加法、减法、移动数据等。

（2） 操作数位置（Operand Location）：指明操作数的地址，告诉 CPU 去哪里找到数据。

（3）.数据类型（Data Type）：指令中操作的数据类型，比如整数、浮点数等。

4. Machine instructions are encoded with what to contain information about the operation required?

机器指令是如何编码来包含操作所需的信息？

- 解答：机器指令通过前缀中的不同位段（bit fields）进行编码，以包含操作所需的信息，比如操作类型、操作数位置、数据类型等。

5. Which of the following is the fastest to execute?

- mov eax, b
- mov a, ebx
- mov eax, ebx
- 问题含义：以下哪条指令的执行速度最快？
- 解答：mov eax, ebx。因为它是寄存器之间的数据传输，数据不需要从内存中加载，速度最快。其他两个选项涉及内存访问，通常比寄存器之间的操作要慢。

6. The following instruction will involve the calculation of ‘effective address’. (T or F)

- mov eax, table[esi]
- 解答：正确（True）。table[esi]中的 table 是基地址，esi 是偏移量。指令会计算基地址加偏移量来得到操作数的有效地址。

7. After the execution of the following instruction, eax will contain the content of message1 inside. (T or F)

- lea eax, message1
- 解答：错误（False）。lea（Load Effective Address）指令不会加载 message1 的内容，而是会将 message1 的地址加载到 eax 寄存器中。它不会直接处理 message1 的值，只会处理地址。
- Q1: 在汇编代码中给 printf 传递两个参数时，第一个参数应该先压入栈中。（True or False）
- 解答：False。调用 printf 时，应该先压入第二个参数，然后是第一个参数。
- Q2: 在汇编中调用 printf 时，传递给它的参数将被 printf 从栈中弹出。（True or False）
- 解答：True。printf 会从栈中弹出传递给它的参数。
- Q3: 在汇编中调用 scanf 时，需要将接收输入的变量地址压入栈中。（True or False）
- 解答：True。scanf 需要接收变量的地址才能将输入数据存储在该变量中。
- Q4: 在 printf 中打印字符串时，使用的转换说明符是什么？
- 解答：%s。
- Q5: 如果在调用 scanf 时有三个整数参数被压入栈中，返回时如何调整寄存器 esp 的值？
- 解答：需要将 esp 增加 12，因为每个整数占 4 字节，三个整数总共 12 字节。

- Q6: 执行 cmp eax, ebx 时会检查零标志的设置。(True or False)
- 解答：True。cmp 指令会根据比较结果设置零标志，如果 eax 和 ebx 相等，则设置零标志为 1。

Q: Conditional jumps in assembly can be used to implement HLL constructs like while, for, and switch. (T or F)

- 答案：True
- 解释：在汇编语言中，条件跳转指令（如 JE、JNE、JG 等）可以用于实现高级编程语言（HLL）中的 while、for、if-else 和 switch-case 语句。这些条件跳转指令根据比较结果改变程序的执行路径，因此可以灵活地模拟各种控制结构。

Q: 'loop' instruction in assembly has a branching effect based upon the value of decremented ECX register. (T or F)

- 答案：True
- 解释：loop 指令在每次执行时会自动递减 ECX 寄存器的值，并根据 ECX 的值决定是否继续循环。当 ECX 变为零时，loop 指令会结束循环；否则，它会跳转到指定的标签继续执行。因此，它具有基于递减后的 ECX 值的分支效果。

Q: Explain what 'LOOPNE label' does.

- 答案：LOOPNE label 指令的作用是：
- 首先，它递减 ECX 寄存器的值。
- 然后，它检查零标志（Z flag），如果 ECX 不为零且零标志为 0（表示之前的比较结果不相等），则跳转到 label。
- 如果 ECX 为零或零标志为 1，则跳过循环。

简而言之，LOOPNE 指令在 ECX 不为零且前一次比较结果不相等的情况下继续循环。

Q: Explain what the following instructions do.

指令：

CMP EAX, EBX

LOOPNE label

- 解释：

1. CMP EAX, EBX：比较寄存器 EAX 和 EBX 的值。它根据比较结果设置 CPU 的标志寄存器（如零标志 Z、符号标志 S 等）。

2. LOOPNE label：递减 ECX 寄存器的值。如果 ECX 不为零且零标志为 0（表示 EAX 和 EBX 不相等），则跳转到 label。否则，程序继续执行下一条指令。

1. A subroutine can be called at various moments from different places in the main program. (T or F)

- 答案：True
- 解释：一个子程序可以在主程序的不同位置、不同时间多次调用。程序中任何地方都可以调用同一个子程序，这就是子程序的灵活性和复用性。例如，函数 average() 可以在不同条件或时刻被多次调用。

2. A subroutine can be called by itself. (T or F)

- 答案：True
- 解释：子程序是可以自我调用的，这种行为叫做递归（Recursion）。递归在编程中非常常见，用来解决诸如阶乘、斐波那契数列等问题。每次递归调用时，返回地址会存储在栈中，保证递归调用结束后，能够返回到正确的地方。

3. How many return addresses can you store using a stack?

- 解释：理论上，你可以在栈中存储无限多个返回地址，但实际受限于栈的大小。栈是有限的，受内存大小的限制。如果函数嵌套调用过多，可能会导致栈溢出（Stack Overflow）。

4. How many nested calls can you make within a program?

- 解释：你可以进行多次嵌套调用，但嵌套的数量取决于栈的大小。每一次调用都会占用栈空间（存储返回地址和局部变量等），当栈空间耗尽时，会导致栈溢出。因此，嵌套调用的次数是有限的，实际情况中可以处理数千甚至数万次的嵌套调用，但具体取决于栈的大小和每次调用占用的空间。

5. How does the computer know when to return from a subroutine?

- 解释：当程序调用一个子程序时，返回地址会被压入栈中。当子程序执行结束并遇到 RET 指令时，栈顶的返回地址会被弹出，并加载到程序指针中，CPU 会跳转到该返回地址处继续执行主程序。因此，计算机通过栈中的返回地址知道从子程序返回的位置。

6. What happens when a ‘CALL’ instruction is executed?

- 解释：当执行 CALL 指令时，以下步骤会发生：

1. 返回地址压栈：当前程序指针（EIP）的值（即调用指令的下一条指令的地址）被压入栈中。

2. 跳转到子程序：程序指针被更新为子程序的起始地址，CPU 开始执行子程序。

7. What happens when a ‘RET’ instruction is executed?

- 解释：当执行 RET 指令时，以下步骤会发生：

1. 弹出返回地址：栈顶的返回地址被弹出。

2. 跳转到返回地址：返回地址被加载到程序指针中，CPU 跳转到返回地址处，继续执行主程序。

1. Q: When is value parameter good for?

- 值参数适用于当你希望在函数调用中传递的参数值保持不变，不被子程序或函数修改的情况。因为值参数将实参的副本传递给子程序，子程序对该副本的操作不会影响到原始数据。

- 适用场景：例如传递数值、常量、局部变量等，它们在函数内部不需要被修改。

2. Q: When is reference parameter good for?

- 引用参数适用于当你希望在函数中修改传递的实参内容的情况。引用参数将变量的地址传递给子程序，子程序可以通过引用对原始变量进行操作。

- 适用场景：例如需要在函数中修改传递的数组、指针或者对象的值，特别适用于需要减少内存复制和提高效率的场合。

3. Q: Why multiple stack frames can coexist at the same time?

- 多个栈帧能够同时存在是因为每次子程序调用都会在栈中创建一个新的栈帧。这些栈帧彼此独立，栈指针 ESP 和基址指针 EBP 会在不同的栈帧之间切换，因此多个栈帧可以共存，保证嵌套调用时每个子程序都有自己独立的栈空间。

4. Q: What type of data are stored under a stack frame?

栈帧中存储的数据包括：

- 函数参数（从主程序或其他子程序传递的参数）。

- 返回地址（子程序执行完毕后返回的地址）。

- 基址指针 EBP（用于保存前一个栈帧的基址）。

- 局部变量（当前子程序中的局部变量）。

5. Q: What happens when a ‘CALL …’ instruction is executed?

- 当 CALL 指令执行时，会进行以下操作：

  将当前的指令指针 EIP（即函数返回的地址）压入栈中。

跳转到被调用函数的地址，开始执行被调用的函数。

6. Q: What happens when a ‘RET’ instruction is executed?

- 当 RET 指令执行时：

从栈中弹出返回地址。

将返回地址加载到指令指针 EIP 中，程序跳转到这个返回地址处，继续执行调用函数之后的代码。

1. Recursive Procedure Exit Condition

1. 递归程序退出条件

- A recursive procedure will typically provide an exiting condition. (T or F)

递归过程通常会提供退出条件。(T 或 F)

- Answer: True •答案：正确

Recursive functions must have a base case to prevent infinite recursion.

递归函数必须有一个基本情况，以防止无限递归。

2. Divide-and-Conquer Step

2. 分治法步骤

- A recursive procedure will typically have a divide-and-conquer step. (T or F)

•递归过程通常有一个分治步骤。(T 或 F)

- Answer: True •答案：正确

Recursive algorithms often break problems into smaller subproblems, solve them recursively, and then combine results.

递归算法通常将问题分解为更小的子问题，递归地解决它们，然后组合结果。

3. Side Effect of the ‘Multiply’ Procedure

3. “乘法”程序的副作用

- What is the side-effect of the procedure ‘multiply’?

•“乘法”的副作用是什么？

- Answer:

The ‘multiply’ procedure modifies values in the registers (eax and edx) to store the multiplication result. The original values are overwritten.

乘法过程修改寄存器（eax 和 edx）中的值来存储乘法结果。原始值将被覆盖。

4. Iterative vs Recursive Implementation

4. 迭代 vs 递归实现

- A recursive procedure can always be re-implemented using iteration without recursion. (T or F)

•递归过程总是可以使用迭代而不需要递归来重新实现。(T 或 F)

- Answer: True •答案：正确

Recursive algorithms can be converted into iterative solutions by using explicit stacks or loops.

递归算法可以通过使用显式堆栈或循环转换为迭代解。

1. Addition for n -digit numbers represented by 10’s complement is based on addition modulo 10^n ?”

1. 以 10 的补数表示的 n 位数的加法是基于对 10^n 取模的加法。

- 答案：True。

n -位数的加法基于 模运算 10^n

2.How is Overflow Detected?

2.如何检测溢出？

溢出检测规则：

- 当两个数的符号相同，而结果的符号与加数符号不同，则发生溢出。

3. Can Overflow Result from Subtractions?

3. 减法会导致溢出吗？

与加法类似：若两个操作数的符号相反，而结果符号与被减数 A 不同，则发生溢出

4. An int type under Java encodes integers under this range: -2^{32} — 2^{32} - 1?

4. Java 中的 int 类型在以下范围内编码整数：-2^{32}-2^{32}- 1？

实际表示范围是从 -2^{31} 到 2^{31} - 1

1. “Under the IEEE 754 standard, how many bits are required to specify the sign of the magnitude?”

答案：1 位

2. “Under the IEEE 754 standard, how many bits are required to specify the sign of the exponent?”

指数是通过 偏移编码（Excess-n） 的方式存储的，无需专门的一位来存储

答案：不需要额外的位

3. “Under the IEEE 754 standard, how many bits are required to specify the decimal point position?”

小数点的位置完全由尾数的规范化和指数决定，不需要额外的位来表示。

答案：不需要额外的位

4. “Does IEEE standard 754 provide a specification for NaN?”

- 指数部分 E_{\text{stored}} = 255（单精度）或 E_{\text{stored}} = 2047（双精度）。
- 尾数部分 不全为 0（否则是无穷大）。

答案：是的，提供了具体的规范

5. “Under the IEEE 754 standard for single-precision floating point format, what type of excess notation is used for exponent specification?”

存储指数范围：0 到 255。

- 实际指数范围：-126 到 +127。

答案：Excess-127

6. “Under the IEEE 754 standard for double-precision floating point format, what type of excess notation is used for exponent specification?

答案：Excess-1023

问题 1：

- 问：计算机系统用哪种编码方案来表示字母、数字和特殊字符？
- 选项：
- a. Magnetic（磁性）。
- b. Analog（模拟）。

- c. Binary（二进制）。
- d. Optical（光学）。
- 正确答案：c. Binary（二进制）。

问题 2：

- 问：以下哪项是易失性存储器？
- 选项：
- A. ROM。
- B. RAM。
- C. DVD。
- D. Hard Disk。
- 正确答案：B. RAM。

问题：以下哪项具有最佳访问速度？

- A. Register（寄存器）
- B. Cache（缓存）
- C. DVD
- D. RAM

答案：A. Register（寄存器）

1. 寄存器（Register）：

- 位于 CPU 内部，是最快的存储单元。
- 它的访问速度极快，因为数据直接存储在处理器核心中，不需要额外的访问路径。
- 主要用于临时存储 CPU 计算中需要的数据。

2. 缓存（Cache）：

- 次于寄存器，位于 CPU 内部或非常接近处理器。
- 它的速度快于 RAM，但比寄存器慢。
- 存储频繁使用的数据，以减少对 RAM 的访问。

3. RAM：

- 相较于寄存器和缓存，RAM 速度较慢。

- 存储运行中的程序和数据，但它需要通过总线（bus）与 CPU 进行数据交换。

4. DVD

- 属于外部存储设备，速度最慢。
- 用于数据存档或分发，无法直接参与高效计算。

总结： 在存储器层次中，寄存器具有最快的访问速度，其次是缓存、RAM，而 DVD 最慢。

1. 问题：

- 以下哪个具有最快的访问速度？
- A. Register
- B. Cache
- C. Disk
- D. Tape

2. 答案：

- A. Register

Q&A 2 - Which is most expensive?

1. 问题：

- 以下哪个存储器的成本最高（每位成本）？
- A. Register
- B. Cache
- C. Disk
- D. Tape

2. 答案：

- A. Register

问题：

- 给定以下组件，形成一个合理的存储器层次结构，以确保良好的成本和速度折中。
- Cache（缓存）
- Disk（磁盘）

- RAM（主存）

答案：

- 合理的存储器层次结构如下：

1. Cache（缓存）：

- 访问速度最快，成本最高。
- 用于存储经常使用的数据。
- 位于 CPU 和 RAM 之间。

2. RAM（主存）：

- 访问速度次于缓存，成本适中。
- 用于存储当前程序运行所需的数据和代码。

3. Disk（磁盘存储）：

- 访问速度最慢，成本最低
- 用于长期存储大容量数据。

局部性的时间属性（Temporal Locality）

问题：

- 局部性可以是时间局部性，这意味着最近使用过的数据比其他数据更可能被访问。（T 或 F）

答案：

- T（True）

解析：

- 时间局部性：
- 描述程序倾向于多次访问最近使用的数据。
- 例如，在循环中访问的变量会在每次循环迭代中重复被访问。

Q: When you save to this, your data will remain intact even when the computer is

turned off.

- a. RAM（随机存取存储器）
- b. motherboard（主板）

- c. secondary storage device（辅助存储设备）
- d. primary storage device（主要存储设备）
- 答案：
- c. secondary storage device（辅助存储设备，如硬盘、SSD 等）。

问题：哪种数据访问方法会减慢数据检索的过程？

- 选项：
- a. direct access storage（直接访问存储）
- b. sequential storage（顺序存储）
- c. random access storage（随机访问存储）
- 答案：
- b. sequential storage（顺序存储）。

Q: The closed, concentric rings on a diskette are referred to as

磁盘上的封闭同心圆被称为？

- 选项：
- a. grooves（凹槽）
- b. tracks（磁道）
- c. sectors（扇区）
- d. circles（圆）
- 答案：
- b. tracks（磁道）。
- 原因：磁盘上的数据以同心圆形式存储，每一个同心圆称为磁道。
- 问题：当从辅助存储（secondary storage）中检索文件并显示在屏幕上时，以下哪种情况是正确的？
- 选项：
- a. 实际上检索了所需文件的副本并将其放置在桌面上。
- b. 旧版本文件仍保留在辅助存储中。
- c. 文件被发送到 ROM。

- d. 如果检索后没有修改文件内容，原始文件不会被替换
- 答案： a,b,d. 旧版本文件仍保留在辅助存储中。
- 原因：文件检索通常是将文件副本加载到内存或缓冲区中，原始文件仍保存在辅助存储中。

Q1: 什么类型的触发器有非法状态？

- SR 触发器（Set-Reset Flip-Flop）：
- 当 $S = 0, R = 0$ 时，输出 Q 和 $\overline{Q}$ 都为 1，这种状态是非法的。

Q2: 什么类型的触发器允许我们复制数据？

- D 触发器（Data Flip-Flop）：
- 通过输入 D 和时钟信号 CK，可以将输入 D 的值存储到输出 Q 中，从而实现数据的复制。

Q3: 哪种电路具有“记忆”功能？

- 时序逻辑电路：
- 包含触发器（如 SR、D、T、JK 触发器）的电路具有“记忆”功能。
- 它们能够记住输入信号的历史状态。

总结

- T 触发器：用于计数器设计。（多个 T 触发器级联可以实现二进制计数）
- D 触发器：用于数据同步与存储，是寄存器的核心元件。
- JK 触发器：功能最强大，适合多种应用。
- SR 触发器：简单但受限于非法状态，需要注意设计约束。